

Assignment III - Valgrind & Pytorch profiler

111550129, 林彦亨

Hardware Information

Component	Spec
GPU	NVIDIA GeForce RTX 4060 Laptop, 8 GB VRAM
CPU	Intel Core i9-14900HX
RAM	32 GB
Disk	WD XS1000 External SSD, 931.5 GB (USB 3.2)
Path	Local

Q1.(30%) Progressive Offload Sweep

1) (10%) Tabulate total throughput and peak gpu mem for the 5 datapoints.

Ans:

Setting	Weight Distribution	Total Throughput (token/s)	Peak GPU Mem (GB)
(1)	100% GPU	239.008	2.806
(2)	50% GPU + 50% CPU	32.861	1.681
(3)	50% GPU + 50% Disk	17.851	1.712
(4)	50% CPU + 50% Disk	13.902	0.587
(5)	100% Disk	9.764	0.587

2) (20%) Complete the table below:

Ans:

Boundary	Ratio	Dominant Bottleneck (PCIe / Disk)
(1)→(2)	$(1) / (2) = \underline{7.27\times}$	PCIe
(2)→(3)	$(2) / (3) = \underline{1.84\times}$	Disk
(3)→(4)	$(3) / (4) = \underline{1.28\times}$	PCIe
(4)→(5)	$(4) / (5) = \underline{1.42\times}$	Disk

After filling in the table, answer:

- a) Within-class comparison: Are the two PCIe boundary ratios close to each other? What about the two Disk boundary ratios? If there's a gap, is the front-pair ratio larger or the back-pair ratio larger? Why?

Ans:

The two PCIe boundaries show significantly different ratios: (1)→(2) yields $7.27\times$, while (3)→(4) yields only $1.28\times$, meaning the front-pair ratio is much larger. In the (1)→(2) transition, weights move from being entirely on GPU (zero PCIe overhead) to having half fetched across the PCIe bus from CPU RAM, resulting in a dramatic throughput drop. In contrast, at the (3)→(4) boundary, Disk I/O is already the dominant bottleneck, so the additional PCIe transfers are largely hidden behind the existing disk latency, producing a much smaller ratio.

The two Disk boundaries are closer to each other: (2)→(3) yields $1.84\times$ and (4)→(5) yields $1.42\times$, with the front-pair ratio slightly larger. Both transitions replace 50% of weights from a faster tier (CPU RAM) with Disk. The front-pair ratio is larger because the baseline in (2) is higher — CPU RAM has significantly greater bandwidth than Disk, so the degradation when switching to Disk is more pronounced.

- b) Cross-class within-pair comparison: In the front pair (1)→(2) PCIe vs (2)→(3) Disk, and in the back pair (3)→(4) PCIe vs (4)→(5) Disk, within each pair, which has the larger ratio — PCIe or Disk? Explain using bandwidth orders of magnitude.

Ans:

Front pair: PCIe ratio is larger.

In the front pair, the PCIe impact dominates because (1)→(2) introduces PCIe transfers from a zero-overhead baseline, causing a massive throughput drop. The Disk impact in (2)→(3) is comparatively small since PCIe is already the bottleneck.

Back pair: Disk ratio is larger.

In the back pair, the two ratios converge. At this stage the system is already heavily reliant on offloading, and the bandwidth gap between CPU RAM (PCIe, ~ 16 GB/s) and Disk (NVMe, ~ 3 GB/s) begins to show more evenly in the overall latency. Since Disk bandwidth is lower, its boundary ratio ends up slightly larger than the PCIe one.

Q2. (20%) Batch Size and I/O Amortization

- 1) (10%) Tabulate wall time, throughput, peak gpu mem for the three batches (1 / 4 / 16).

Ans:

Batch Size	Wall Time (s)	Total Throughput (token/s)	Peak GPU Mem (GB)
b=1	6.508	2.459	0.469
b=4	6.555	9.764	0.587
b=16	6.482	39.496	1.050

- 2) (5%) Compute the throughput ratios between adjacent batches: $b4/b1 \approx ?$, $b16/b4 \approx ?$. How do the measured ratios compare to the theoretical $4\times$ (since batch $\times 4$)? Cross-reference how wall time changes with batch to explain what you observe.

Ans:

Adjacent	Batches Measured	Ratio Theoretical Ratio
b4 / b1	$9.764 / 2.459 = 3.97\times$	$4\times$
b16 / b4	$39.496 / 9.764 = 4.04\times$	$4\times$

The measured ratios are very close to the theoretical $4\times$. This can be explained by observing that total latency remains nearly constant across all three batch sizes (approximately 6.5 seconds). Under 100% Disk offload, each generation step must load the entire model weights (~2.4 GB) from disk regardless of batch size — this I/O cost dominates and is effectively fixed. Increasing the batch size simply processes more prompts within the same I/O window, so throughput scales almost linearly with batch size, closely matching the theoretical $4\times$ per step.

- 3) (5%) Looking at how peak GPU mem changes with batch, can batch be scaled indefinitely? Why?

Ans:

Although throughput scales nearly linearly with batch size, batch size cannot be increased indefinitely. As batch size grows, the KV cache and activation sizes grow proportionally, consuming more GPU VRAM. In our results, peak GPU memory increases from 0.469 GB (b=1) to 1.050 GB (b=16). With a fixed VRAM budget of 8 GB on the RTX 4060 Laptop, a sufficiently large batch will cause the KV cache to exceed available VRAM, resulting in an out-of-memory (OOM) error. Therefore, batch size is upper-bounded by available GPU memory.

Q3. (20%) Weight Compression and Tier Interaction

- 1) (10%) Tabulate the throughput and peak gpu mem under (α) and (β), with and without compression.

Ans:

Scenario	Compress	Total Throughput (token/s)	Peak GPU Mem (GB)
(α) 100% GPU	No	239.008	2.806
(α) 100% GPU	Yes	47.504	1.103
(β) 100% Disk	No	9.764	0.587
(β) 100% Disk	Yes	10.540	0.469

- 2) (10%) Compare the throughput change (direction + magnitude) when --compress-weight is enabled in (α) vs (β). From your data, answer:
- a) What cost does compression reduce? What cost does it add? Which dominates in each scenario, and how does that determine whether throughput speeds up or slows down?

Ans:

Enabling --compress-weight quantizes weights from FP16 to 4-bit, achieving a 4 $\times$ reduction in weight size. This reduces two costs: the amount of data that needs to be transferred from storage (either Disk or CPU RAM) to GPU, and the GPU VRAM footprint of the weights. However, compression introduces an additional cost: every time a weight is used, it must first be dequantized back to FP16 before the GPU can perform computation, adding extra compute overhead per layer.

In scenario (α) 100% GPU, all weights already reside in GPU VRAM, so there is no I/O transfer to benefit from. The only effect of compression is the added dequantization overhead. As a result, throughput drops significantly from 239.008 to 47.504 token/s ($\sim 5\times$ slowdown), confirming that dequantization cost dominates when I/O is not the bottleneck.

In scenario (β) 100% Disk, weights must be loaded from disk on every generation step, making Disk I/O the dominant bottleneck. Compression reduces the amount of data read from disk by $\sim 4\times$, substantially lowering I/O cost. Although dequantization overhead is added, the time saved on Disk I/O outweighs it, resulting in a modest throughput increase from 9.764 to 10.540 token/s ($\sim 1.08\times$ speedup). Here, I/O reduction dominates over dequantization cost.

- b) Why is the magnitude of the change (whether speedup or slowdown) not close to the $4\times$ compression ratio?

Ans:

In scenario (α), the $\sim 5\times$ slowdown exceeds what a simple $4\times$ compression ratio would suggest, because dequantization imposes a per-layer compute penalty on top of the normal GPU computation, amplifying the overhead beyond the compression ratio itself.

In scenario (β), the speedup is only $\sim 1.08\times$ despite a $4\times$ reduction in data read from disk. This is because Disk I/O is just one stage in the overall decode pipeline — the pipeline also includes dequantization, GPU matrix computation, and data movement between tiers. Amdahl's Law applies here: even if Disk I/O is accelerated by $4\times$, the other stages remain unchanged and cap the overall speedup. Additionally, the dequantization cost introduced by compression partially offsets the I/O savings, further limiting the net gain.

Q4. (30%) I/O Behavior and Bottleneck Analysis under Disk Offload

1. (15%) Part 1: disk I/O pattern

Ans:

iostat (disk offload)	Value
r/s peak	47 req/s
rkB/s peak	18064 KB/s
rareq-sz peak	384.34 KB/req
%util peak	91.4 %

After filling in the table, classify FlexGen's behavior into one of the following and justify your choice using your experimental data:

- A. Sequential, large blocks: rareq-sz peak ≥ 32 KB (kernel readahead batched consecutive pages into large requests).
- B. Random, small blocks: rareq-sz peak < 32 KB with high r/s (each request independent, kernel can't batch).
- C. Page cache hit: rareq-sz peak ≈ 4 KB and rkB/s peak much lower than your disk's spec (reads don't actually reach the disk).

Ans:

(A) Sequential, large blocks.

The rareq-sz peak of 384.34 KB/req far exceeds the 32 KB threshold, indicating that the Linux kernel's readahead mechanism is successfully batching consecutive pages into large requests. This is consistent with FlexGen's access pattern: weights are

stored as numpy arrays on disk and loaded layer by layer in a fixed sequential order. Because the access pattern is predictable and contiguous, the kernel prefetches and merges multiple pages into a single large I/O request, resulting in a low r/s (47 req/s) but high per-request size. The %util of 91.4% confirms that the disk is nearly saturated during inference, consistent with Disk I/O being the dominant bottleneck in this configuration.

2. (15%) Part 2: Stage timings in the two scenarios

Ans:

Scenario	load_weight (per-layer, sec)	compute_layer_decoding (per-batch, sec)	load / compute ratio
(1) Baseline	0.000020 s	0.000729 s	0.027×
(5) Disk offload	0.008222 s	0.001066 s	7.71×

After filling in the table:

- a) Compare compute_layer_decoding across baseline and disk offload — what do you observe? What does this tell us? (Hint: what does offload change? Is it related to GPU compute volume?)

Ans:

The compute_layer_decoding time is nearly identical between the two scenarios: 0.000729 s for baseline and 0.001066 s for disk offload, a difference of only $\sim 1.46\times$. This is expected because offloading only changes where weights are stored, not the amount of computation the GPU must perform. Every token generation still requires the same matrix multiplications across all 50 layers regardless of the weight storage tier. The small difference may be attributed to minor pipeline scheduling overhead introduced by the offloading mechanism.

- b) Compare load_weight across the two scenarios — by how many orders of magnitude do they differ? What aspect of the system does offload change, based on this?

Ans:

The load_weight time differs by approximately $411\times$ between the two scenarios (0.000020 s vs. 0.008222 s), spanning more than two orders of magnitude. In the baseline, all weights already reside in GPU VRAM, so load_weight is effectively just a pointer dereference with negligible cost. In the disk offload scenario, every layer's weights must be read from disk on

each generation step, bottlenecked by the disk's read bandwidth (~ 17.6 MB/s as measured in Part 1). This causes `load_weight` to dominate the entire decode pipeline, as reflected by the load/compute ratio jumping from $0.027\times$ to $7.71\times$ — meaning that under disk offload, data movement takes roughly $7.7\times$ longer than the actual GPU computation per layer.